COMSATS UNIVERSITY ISLAMABAD

Sahiwal Campus



Talent Bridge

Java OOP Project Report

Project Title	Talent Bridge (Job Portal)
Submitted By	Muhammad Abdullah Majid, Muhammad Aoun Ali Niaz
Registration No.	Withheld
Submitted To	Ms. Abida Kousar
Course	CSC241 - Object Oriented Programming
Institute	COMSATS University Islamabad, Sahiwal Campus
Submission Date	June 23, 2026

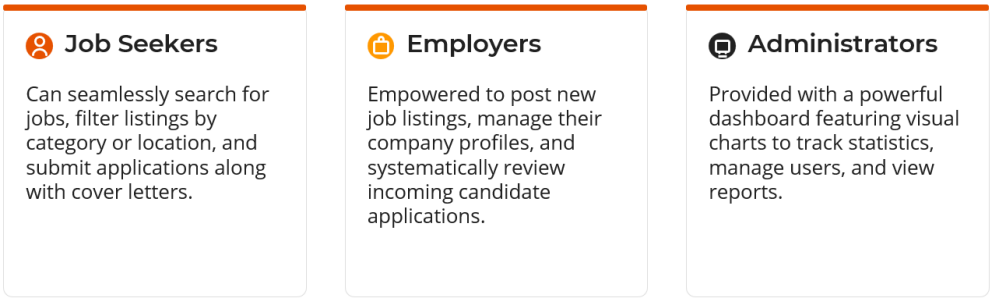
Department of Software Engineering

Table of Contents

1. Introduction & Overview 1
Objective 1
Architecture 1
2. OOP Principles Demonstrated 2
Abstraction: 2
Inheritance: 2
Polymorphism: 3
Encapsulation: 3
Singleton Pattern: 4
Interfaces: 4
Composition: 4
3. Tools & Technologies Used 4
MySQL Database: 5
Users 6
Jobs: 6
Applications: 6
4. Talent Bridge Application 7
Login and Registration Panel 7
Job Seeker Module 8
Employer Module 10
Administrator Module 12
5. Testing 14
6. Conclusion 15
7. Future Enhancements 15
Web & Mobile Migration: 15
In-App Communication: 15
Enhanced Security Measures: 15
Resume Parsing & Generation: 15

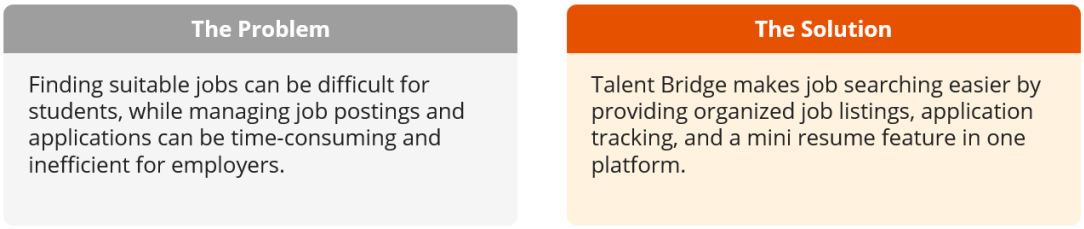
1. Introduction & Overview

In today's fast-paced professional world, efficiently bridging the gap between skilled candidates and organizations seeking talent is more critical than ever. **Talent Bridge** is a comprehensive, desktop-based job portal application designed to address this challenge by providing a streamlined, intuitive, and secure platform for employment networking.



Objective

The primary objective of the Talent Bridge project was to engineer a monolithic client-server architecture capable of supporting multiple distinct user types simultaneously. The system successfully facilitates a dynamic ecosystem comprising three primary roles: **Job Seekers**, who utilize the platform to maintain professional profiles and discover career opportunities; **Employers**, who leverage the system to post job listings and manage candidate applications;



Architecture

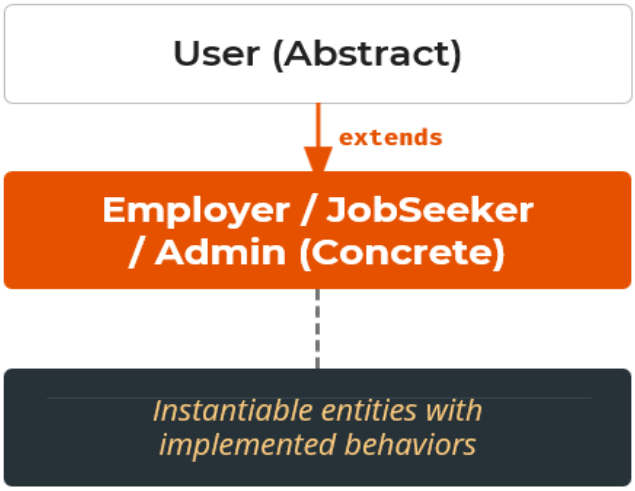
From a technical perspective, the application is built entirely using Core Java (JDK 8+) and relies heavily on the Java Swing framework to deliver a rich, custom-styled Graphical User Interface (GUI). To ensure a modern user experience, the interface departs from standard native look-and-feels, opting instead for custom 2D graphics rendering including rounded panels, dynamic hover animations, and bespoke data visualization charts built from scratch. Data persistence is handled securely via a centralized MySQL database, connected using the JDBC API (mysql-connector-j-8.3.0.jar).

Crucially, the architecture of Talent Bridge serves as a showcase of fundamental OOP methodologies. By utilizing a base abstract User class and extending it across role-specific models, the project heavily leverages **Inheritance** and **Polymorphism** to dynamically render appropriate dashboards upon user login. Data integrity is strictly maintained through **Encapsulation**, while the database connection lifecycle is expertly managed via the **Singleton** design pattern in a centralized DataStore class

2. OOP Principles Demonstrated

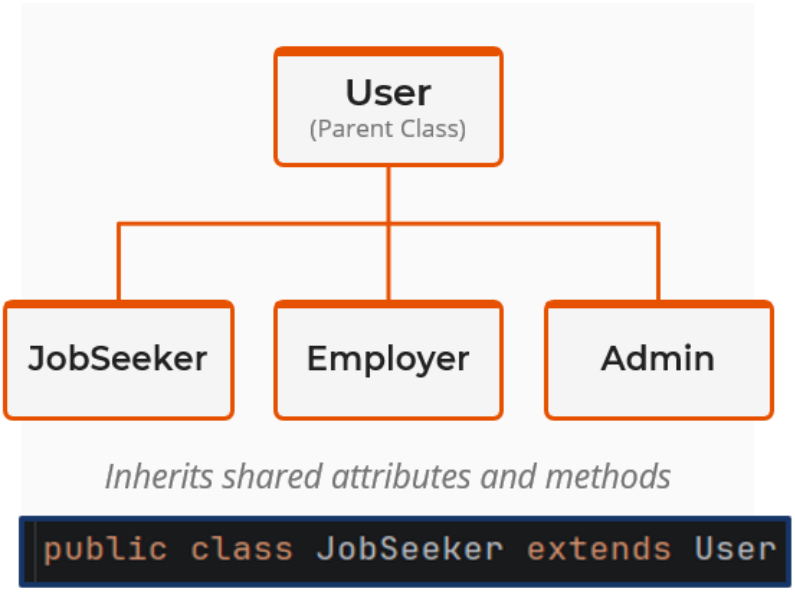
Abstraction:

The project hides complex implementation details and exposes only necessary functionalities. This is primarily achieved through the abstract User class, which defines the core template for all system users without allowing a generic "User" to be instantiated directly.



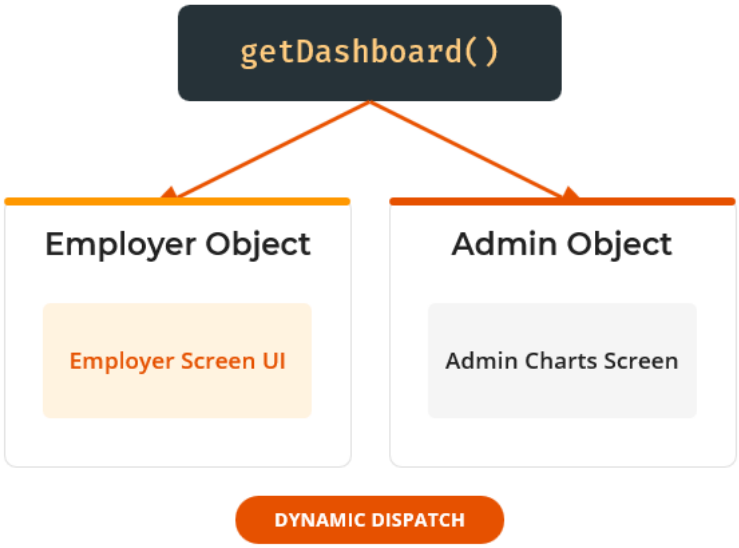
Inheritance:

To promote code reusability and establish a hierarchical relationship, the JobSeeker, Employer, and Admin classes inherit shared attributes (like ID, username, email, and password) and methods from the parent User class, while adding their own role-specific fields (such as companyName for Employers or skills for Seekers).



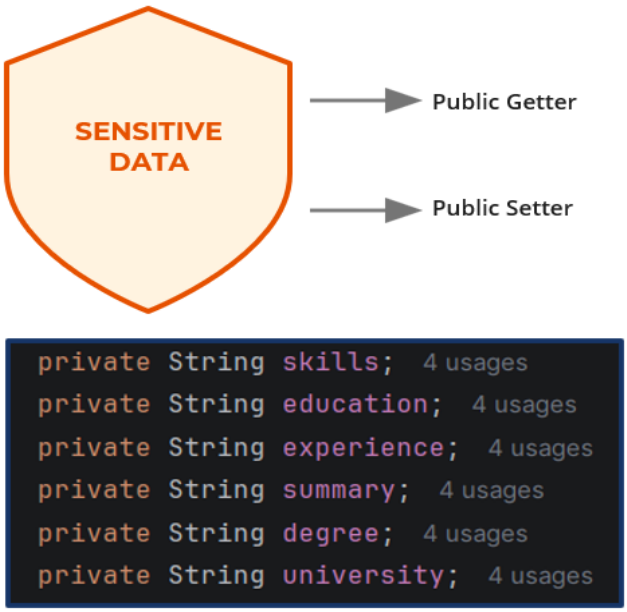
Polymorphism:

The system utilizes polymorphism to treat different user types uniformly while allowing them to exhibit unique behaviors. For example, the getRole() method is overridden in each child class. Additionally, when a user logs in, the TalentBridge main frame dynamically determines whether to render the Seeker, Employer, or Admin dashboard based on the specific object type returned by the database.



Encapsulation:

Data protection is enforced across all model classes (Job, JobApplication, User, etc.) by declaring class variables as private. External classes cannot modify this data directly; instead, they must use public getter and setter methods, ensuring data integrity and validation before any changes are made.



Singleton Pattern:

To prevent the application from opening multiple, conflicting connections to the MySQL database and wasting memory, the DataStore class implements the Singleton pattern. It uses a private constructor and a getInstance() method to guarantee that only one instance of the database manager exists throughout the entire lifecycle of the application.

Interfaces:

The Searchable interface establishes a contract for search behaviors.
The DataStore implements this interface to guarantee that specific filtering methods (like searchJobs, filterByType, and filterByLocation) are consistently defined and available for querying the database.

Composition:

Instead of relying strictly on inheritance for everything, the project builds complex objects by assembling smaller ones. For example, a JobApplication object does not inherit from Job or JobSeeker; rather, it uses their IDs to link them together, and a Job object conceptually "has an" Employer associated with it.

3. Tools & Technologies Used

The **Talent Bridge** application is driven by a robust backend developed entirely in Core Java (JDK 8+). By deeply integrating fundamental Object-Oriented Programming (OOP) paradigms such as Polymorphism, Inheritance, Encapsulation, and Abstraction, the backend logic securely structures user roles, job data, and complex application pipelines. To ensure optimal performance and seamless data manipulation in memory, the backend logic relies heavily on the standard Java Collections Framework specifically utilizing Lists, Maps, and HashMaps paired with the Java 8 Streams API for efficient, functional-style filtering and dynamic data aggregation.

The application’s frontend departs from standard, rigid interfaces by leveraging the Java Swing framework alongside the native Abstract Window Toolkit (AWT). To achieve a highly customized and modern user experience without resorting to external UI libraries, the project extensively utilizes the Graphics2D class. By overriding native paint methods, the interface renders anti-aliased geometric shapes, rounded cards, and bespoke data visualizations like mathematical pie and bar charts entirely from scratch. This seamless, "Single Page Application" feel on the desktop is achieved using Swing's CardLayout to dynamically transition between login screens, registration pages, and role-specific dashboards, all driven by custom mouse and action event listeners.

For reliable data persistence, the project employs a highly normalized MySQL relational database engine. The communication bridge between the Java application and the MySQL server is established through the Java Database Connectivity (JDBC) API using the standard mysql-connector-j driver. To maintain absolute structural integrity, the database schema enforces strict foreign key constraints with cascading deletions. Furthermore, application security is paramount; every database query is executed via JDBC PreparedStatement interfaces. This critical architectural choice inherently protects the system against SQL Injection attacks by safely parameterizing all user inputs before they ever interact with the database engine.

Category	Technology
Programming Language	Java (JDK 8+)
GUI Toolkit	Java Swing & AWT (Graphics2D rendering)
Database	MySQL 8.x
Connectivity	JDBC (mysql-connector-j-8.3.0.jar)

MySQL Database:

The **Talent Bridge** application utilizes a highly normalized **MySQL** database connected securely to the Java backend via the JDBC API. Data persistence is managed across five core tables: a central users table for shared authentication and base profiles, which links out to specialized job_seekers and employers tables to store role-specific data like candidate skills or company sizes. Job listings created by recruiters are stored in a dedicated jobs table, while an applications junction table maps candidates to specific jobs, tracking their real-time hiring status. To ensure absolute data integrity and prevent orphaned records, the schema heavily employs Foreign Keys with ON DELETE CASCADE, ensuring that if a user or job is deleted, all associated data is automatically removed.

Users

id	username	password	email	full_name	phone	city	role
1	admin	[redacted]	[redacted]	System Admin	[redacted]	NULL	ADM
2	abdullah	[redacted]	[redacted]	Muhammad Abdullah	[redacted]	Sahiwal	JOB
3	azaan	[redacted]	[redacted]	Muhammad Azaan	[redacted]	Sahiwal	EMP
14	huzy	[redacted]	[redacted]	Huzaifa	[redacted]	Lahore	JOB
19	rizwan	[redacted]	[redacted]	Rizwan	[redacted]	Sahiwal	JOB
20	ali	[redacted]	[redacted]	Ali	[redacted]	Lahore	JOB
21	ahmed	[redacted]	[redacted]	Ahmed	[redacted]	Lahore	JOB
22	sufyan	[redacted]	[redacted]	Sufyan	[redacted]	Okara	EMP
23	faizan	[redacted]	[redacted]	Faizan	[redacted]	Pakpattan	JOB
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Jobs:

id	employer_id	title	company_name	description	requirements
18	22	Junior Pyton Developer	Apex Limited	Need Frontend Developer	Java
19	3	Mern Fullstack Developer	Super Limited	Fullstack Developer Required	Typescript, Javascri
21	22	Expert HTML Developer	Apex Limited	HTML expert	HTML
23	22	MachineLearning Intern	Apex Limited	ML	ML
NULL	NULL	NULL	NULL	NULL	NULL

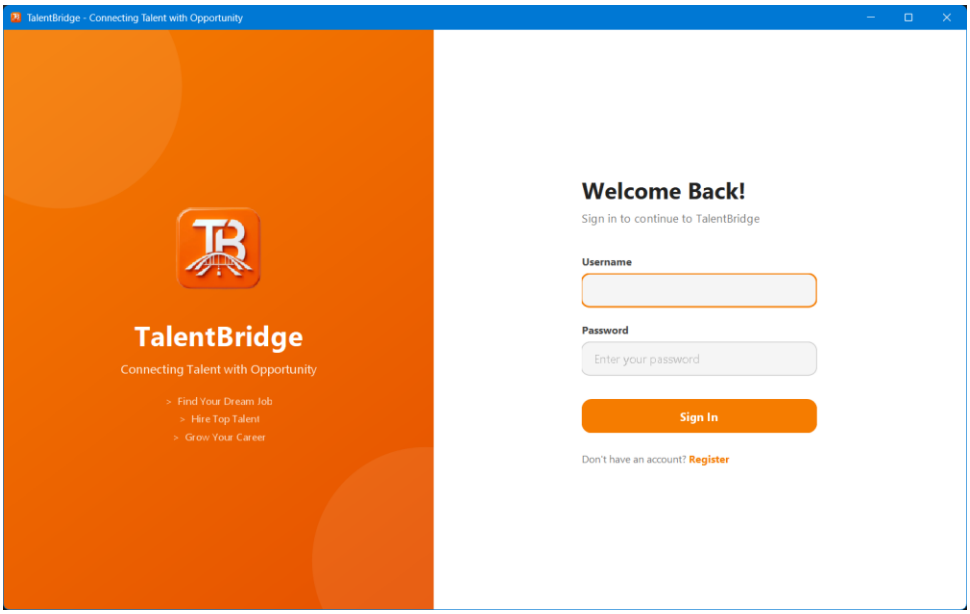
Applications:

id	job_id	seeker_id	cover_letter	status	applied_date
12	18	14	I am very interested in the Junior Fro...	PENDING	2026-06-22 00:04:19
13	19	14	I am very interested in the Mern Fullst...	PENDING	2026-06-22 00:08:51
15	18	2	I am very interested in the Junior Pyt...	PENDING	2026-06-22 15:44:06
16	19	2	I am very interested in the Mern Fullst...	WITHDRAWN	2026-06-22 15:46:01
NULL	NULL	NULL	NULL	NULL	NULL

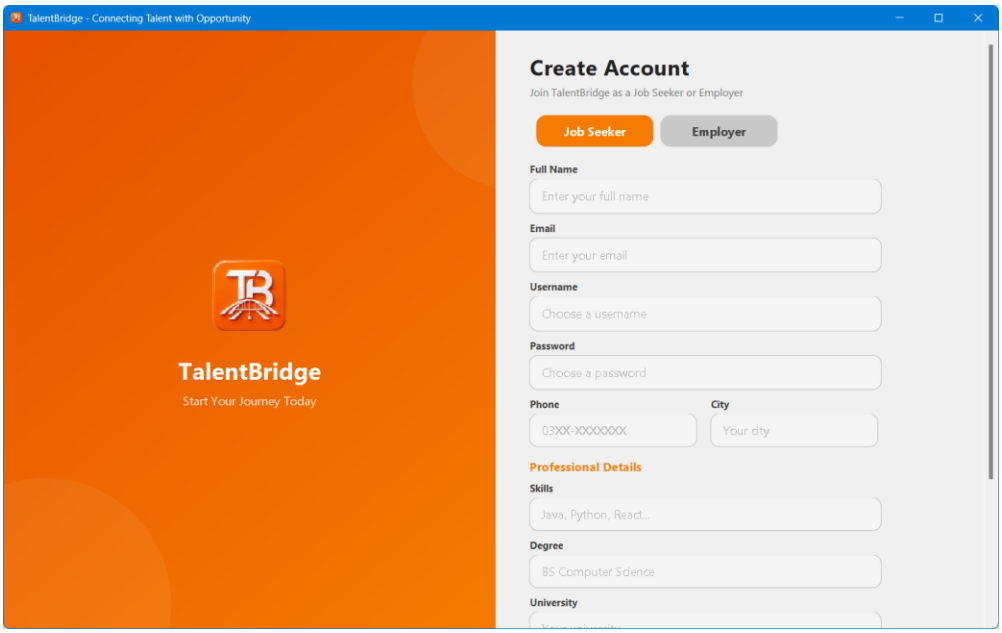
4. Talent Bridge Application

The application is built as a desktop application. TalentBridge.java acts as the main file and uses CardLayout to switch smoothly between screens such as Login, Registration, and different user dashboards. It also keeps the user interface separate from data handling for better organization.

Login and Registration Panel



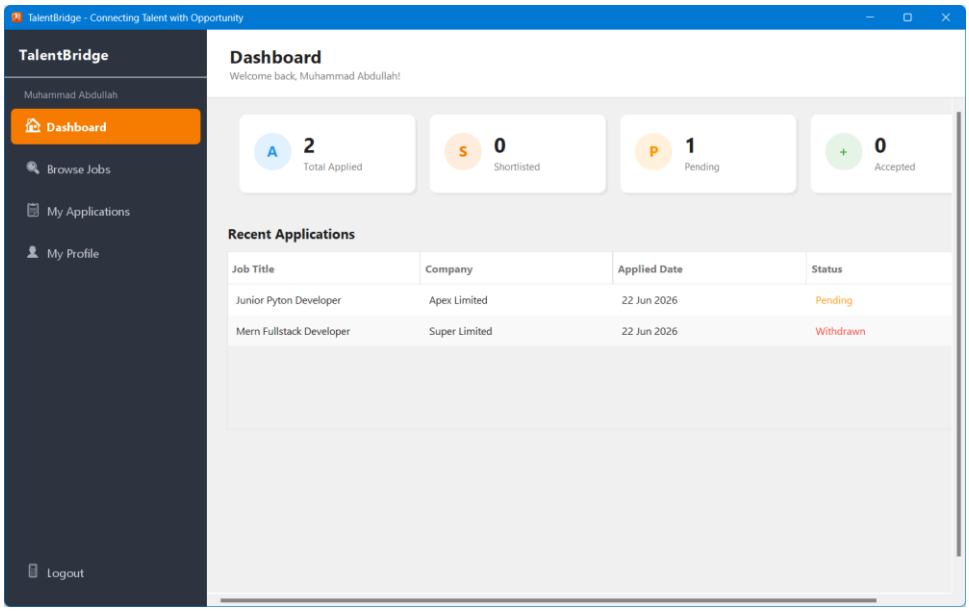
Two types of users can register, JobSeeker & Employeer.



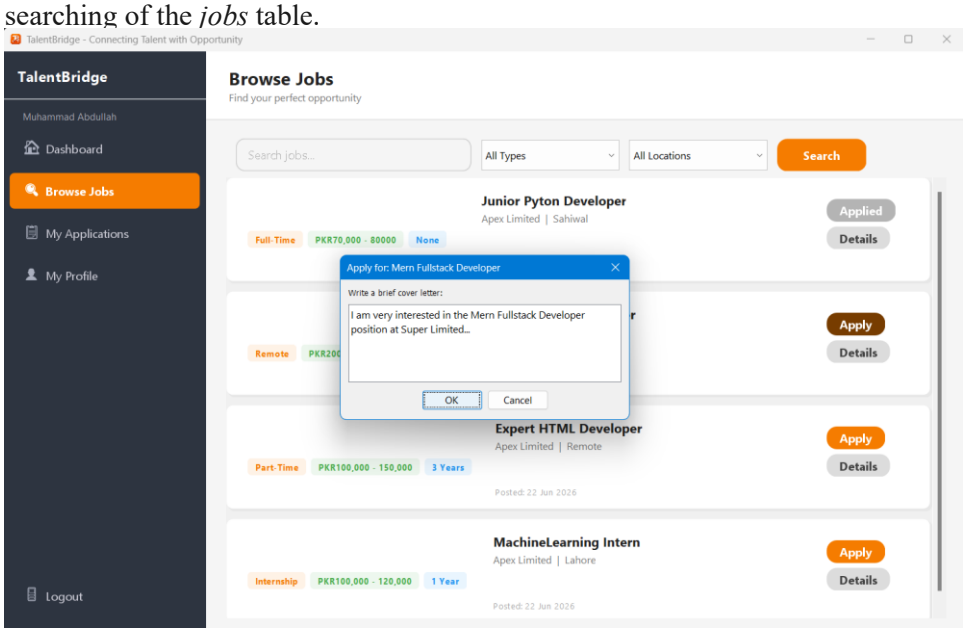
Job Seeker Module

Job Seekers utilize the platform as candidates seeking employment opportunities. They can build and maintain a comprehensive digital professional profile, detailing their education, skills, and experience. Using dynamic search and filtering tools, seekers can easily browse active job listings to find roles that match their criteria. Once they find a suitable position, they can submit an application along with a customized cover letter, track the real-time status of their submissions, and even withdraw applications if they are no longer interested.

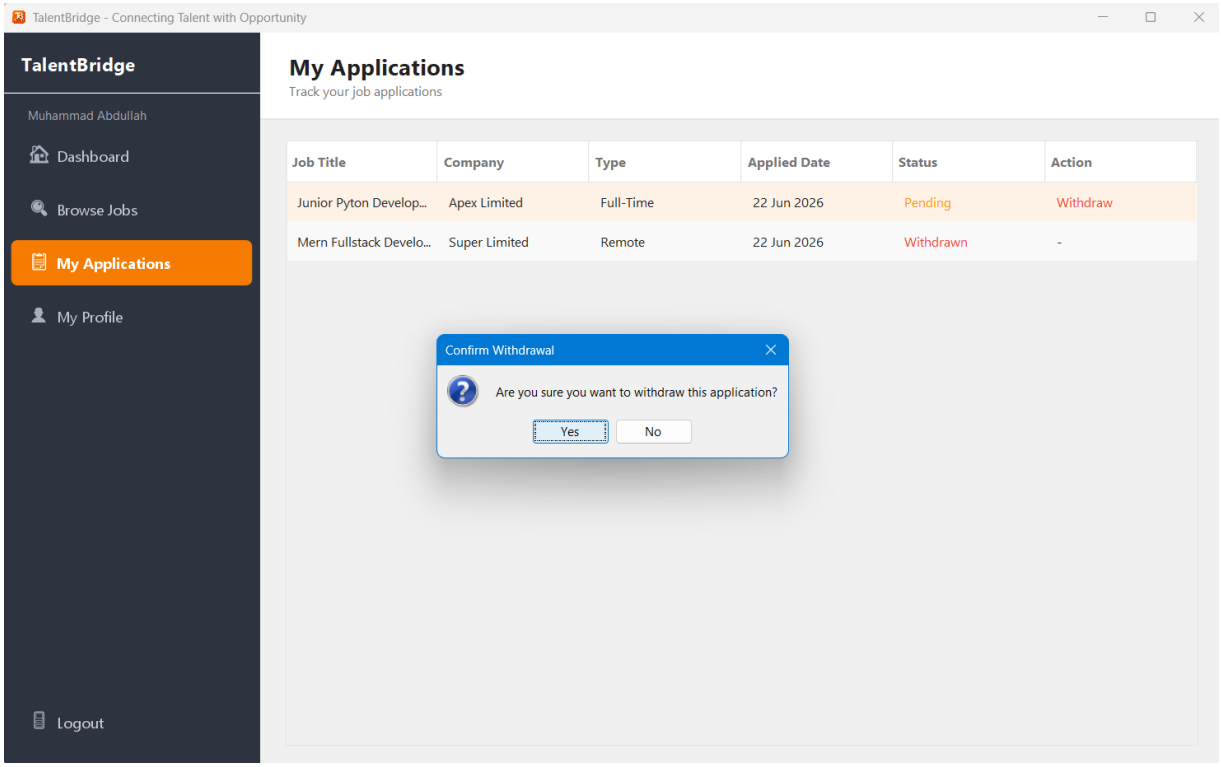
Overview Tab: Displays statistical cards (Total Applied, Shortlisted) and a table of the 5 most recent job applications.



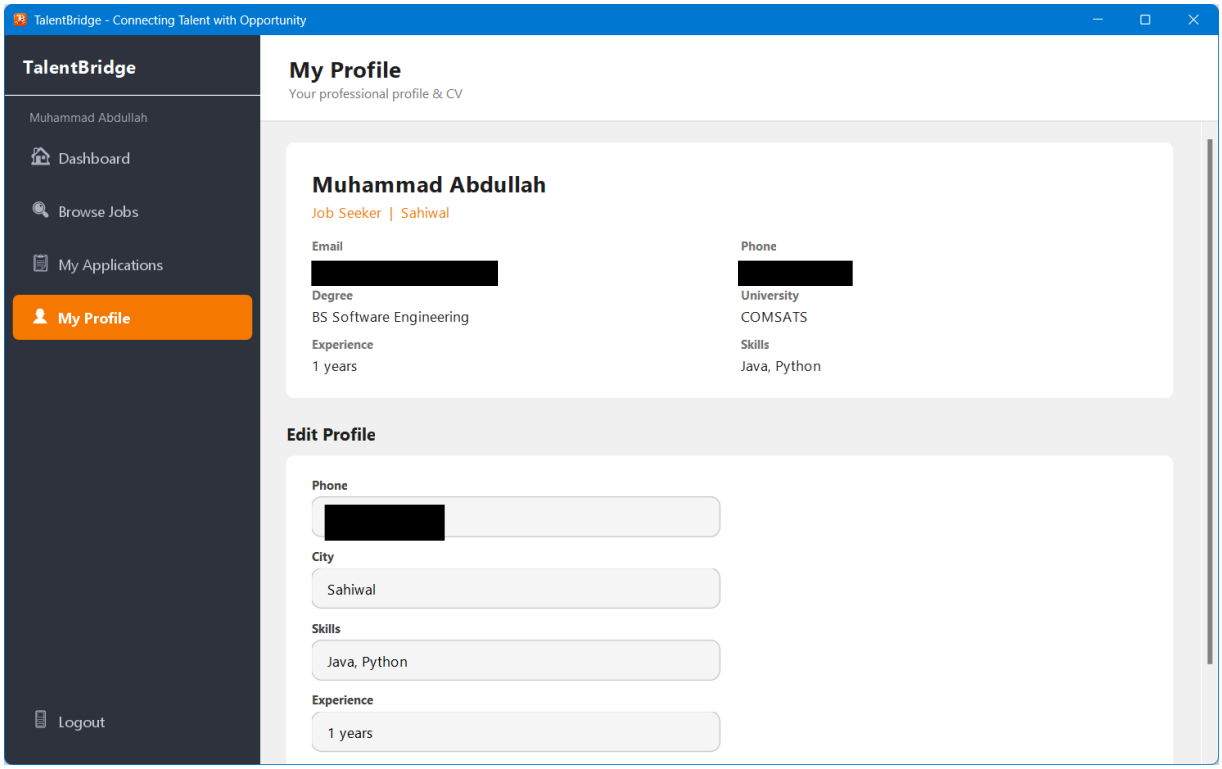
Browse Jobs Tab: Features dynamic filters for job types and locations, allowing real-time searching of the *jobs* table.



My Applications Tab: Provides a full registry of applied jobs with the functionality to withdraw pending applications.



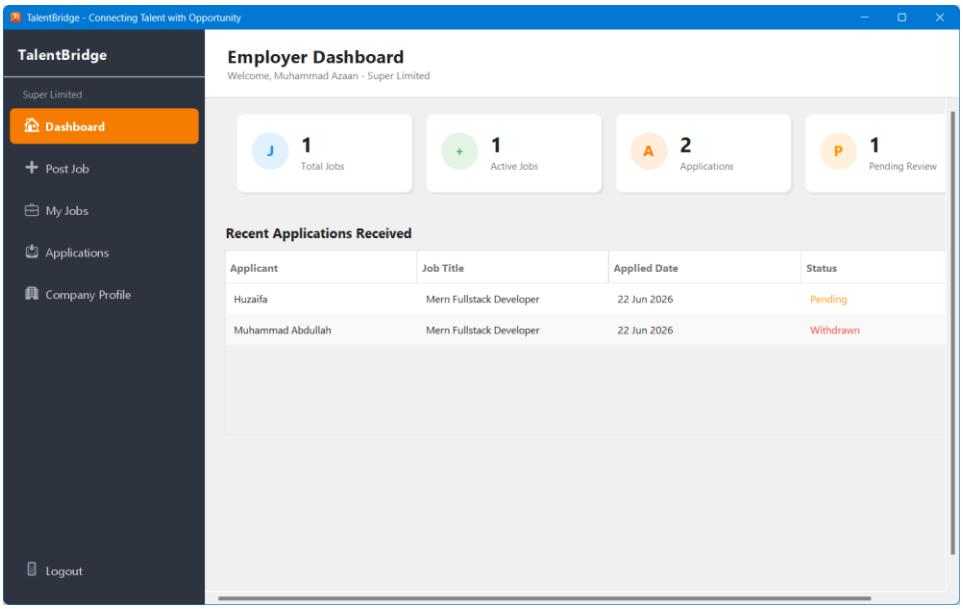
Profile Tab: A digital CV view where users can update their skills, experience, and contact information.



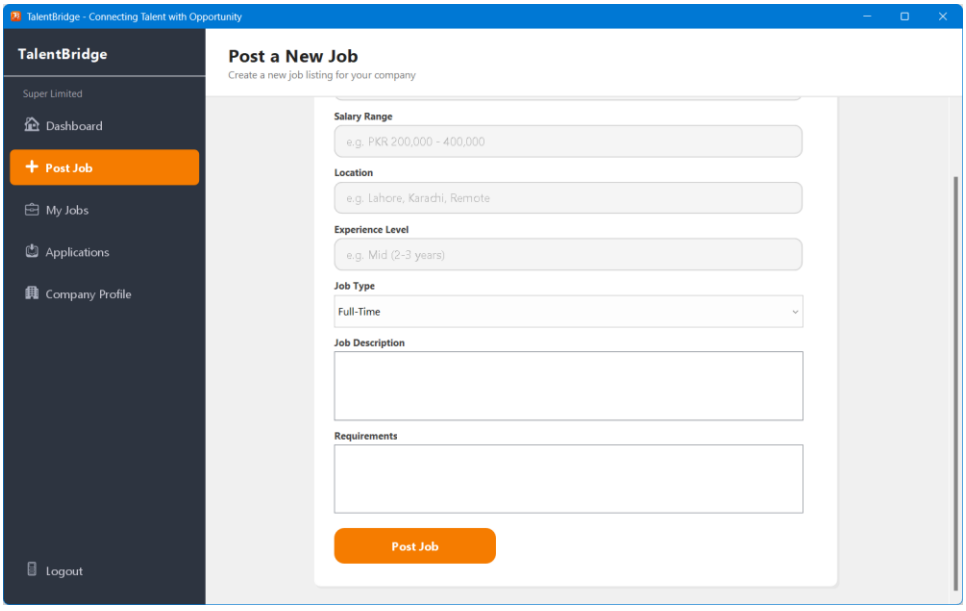
Employer Module

Employers act as the recruiters on the platform, managing their company's public branding and hiring pipelines. They maintain a company profile that highlights their industry, size, and culture to attract top talent. Employers have full control over their job listings, allowing them to create new postings, edit details, and toggle the active status of jobs as their hiring needs evolve. They receive applications directly on their dashboard, where they can thoroughly review candidate profiles and cover letters before updating the application statuses to shortlist, accept, or reject individuals.

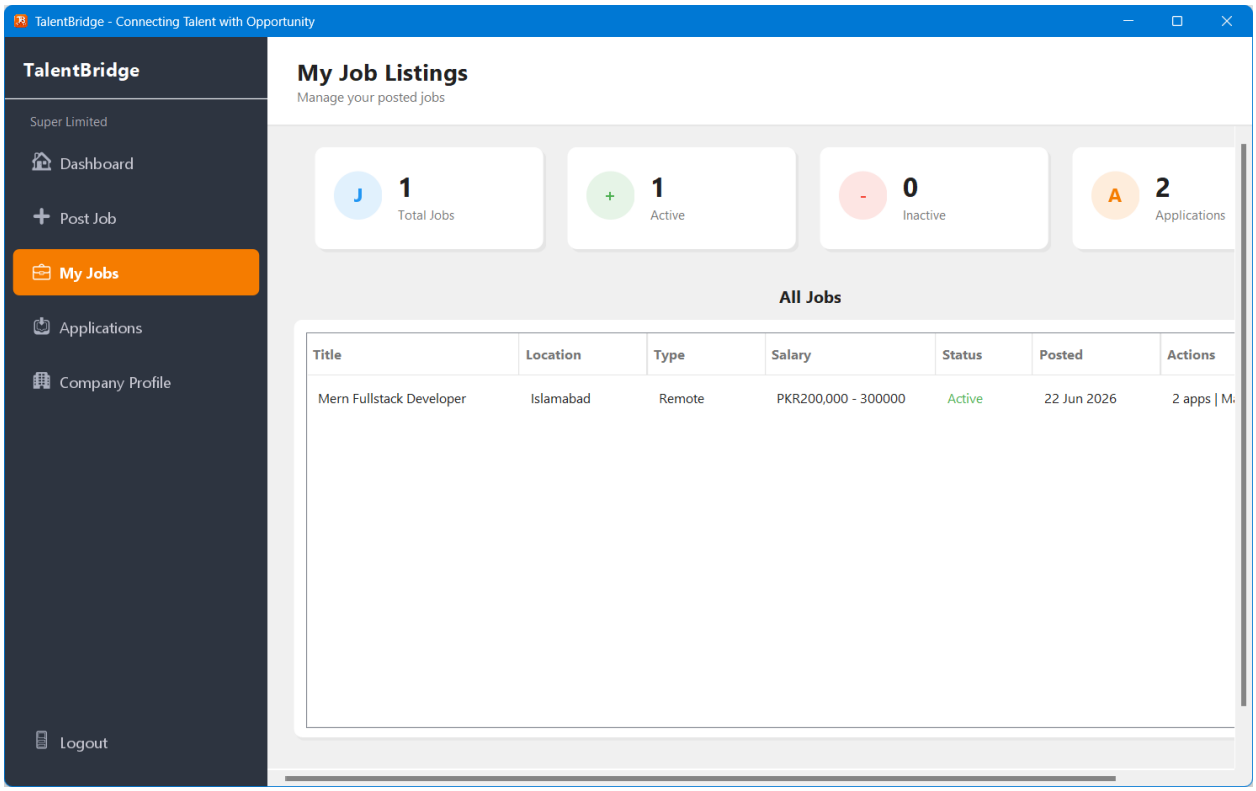
Overview Tab: Recruitment analytics showing total active job listings and pending candidate reviews.



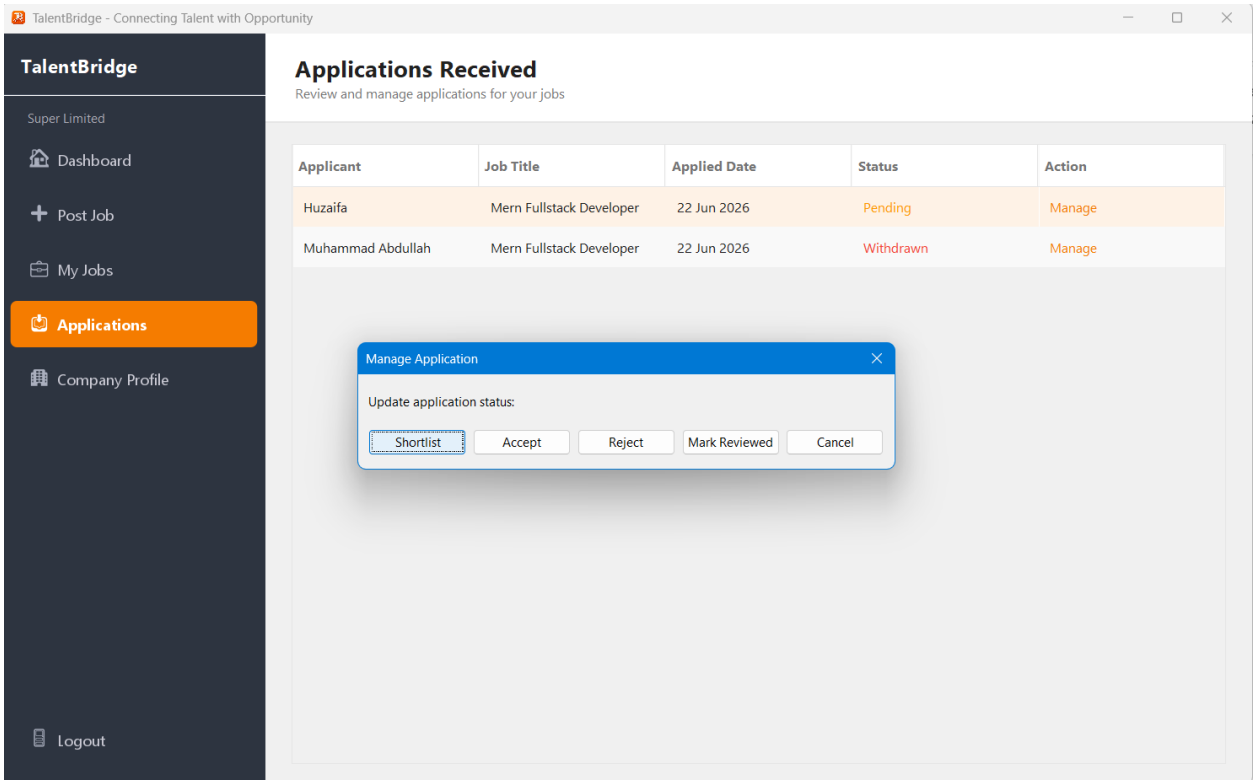
Post Job Tab: A comprehensive form to define job roles, requirements, and salary ranges directly into the database.



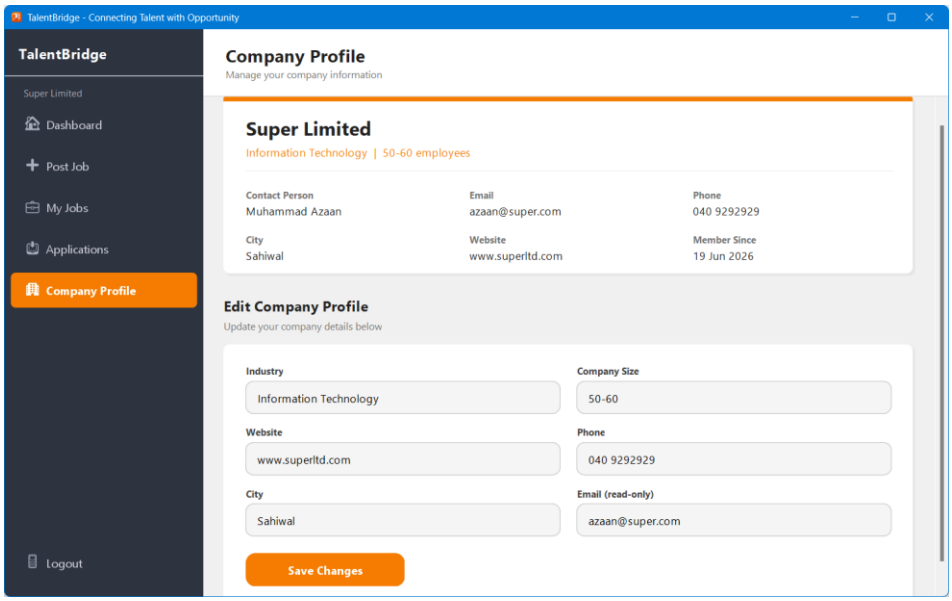
My Jobs Tab: Interface to track posted jobs and toggle their status between 'Active' and 'Inactive'.



Applications Tab: Module for screening applicants, viewing cover letters, and updating application status (Shortlisted, Accepted, Rejected).



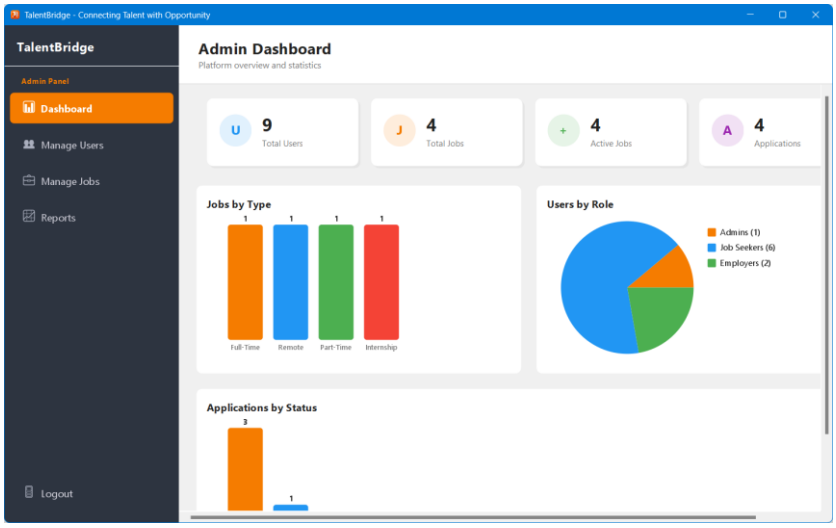
Company Profile: A digital profile where users can view and update their company information



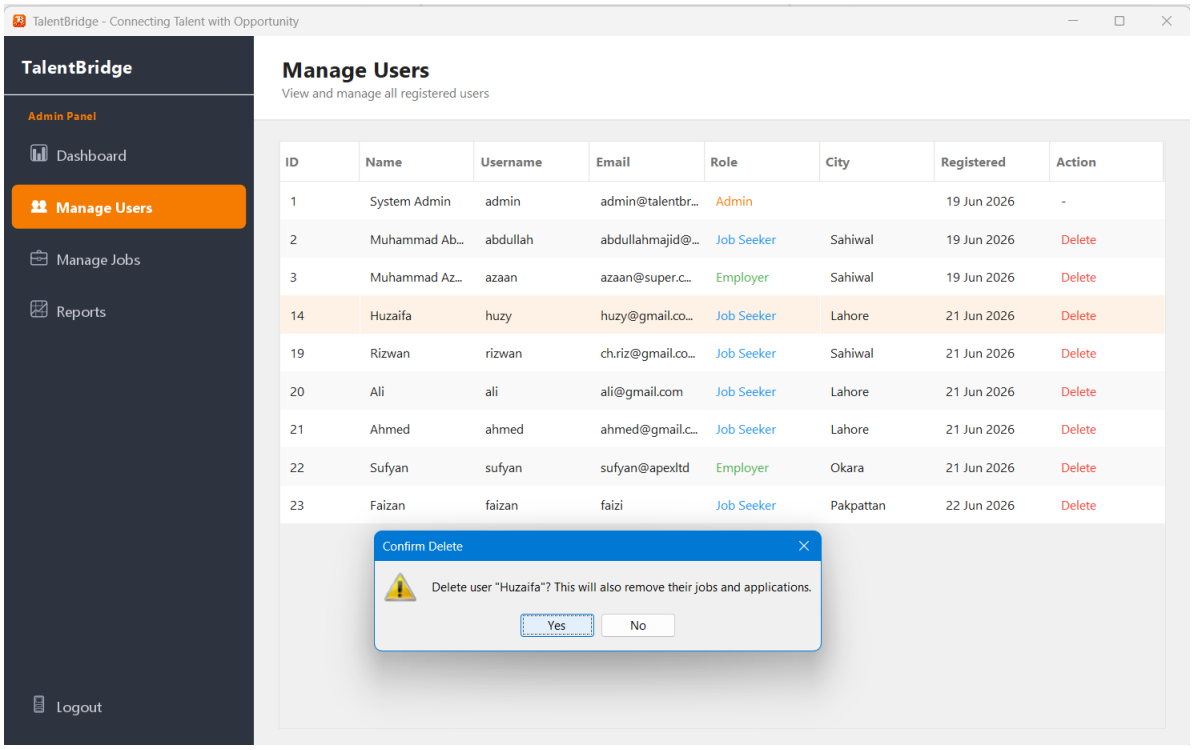
Administrator Module

Administrators serve as the platform managers, ensuring the entire job portal operates smoothly and safely. They possess a high-level analytics dashboard equipped with custom data visualizations to monitor platform health, user engagement, and job trends. Admins hold complete oversight and moderation capabilities, allowing them to view the entire registry of users and active jobs. To maintain a high-quality environment, they wield the authority to forcefully remove inappropriate job listings and delete user accounts that violate the platform's terms of service.

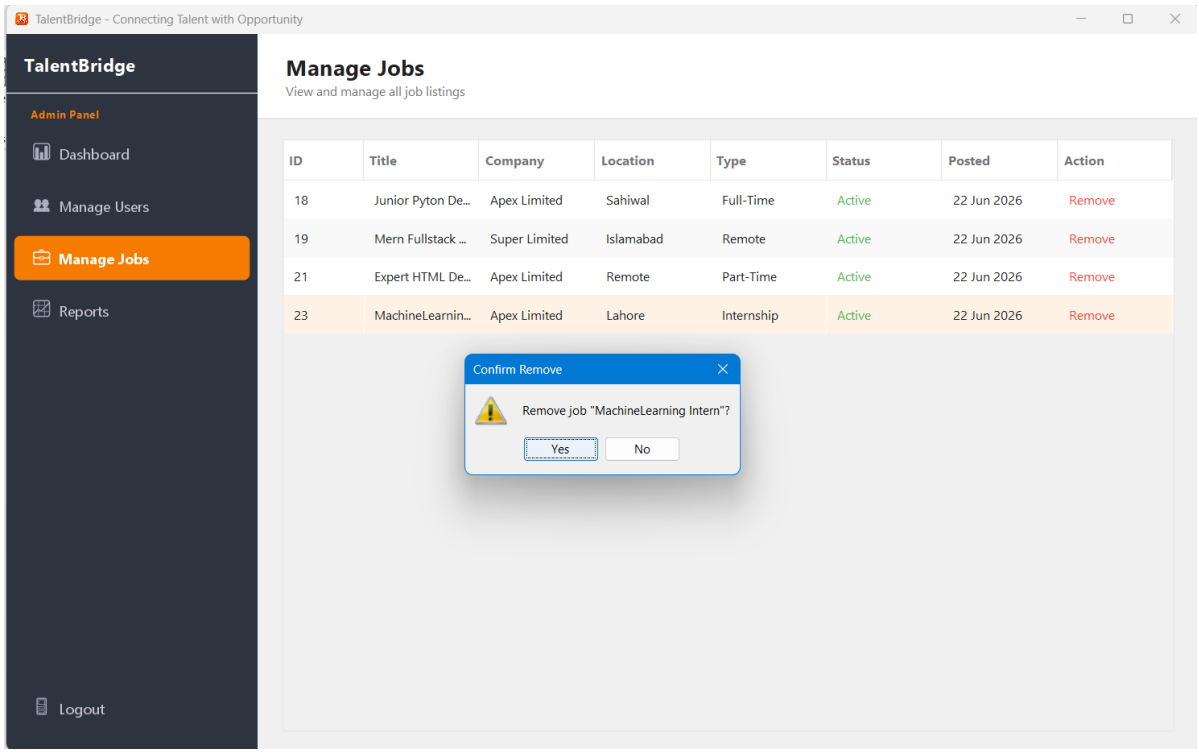
Analytics Dashboard: Displays Pie and Bar charts drawn purely via *Graphics2D*, showing distributions of jobs by type and users by role.



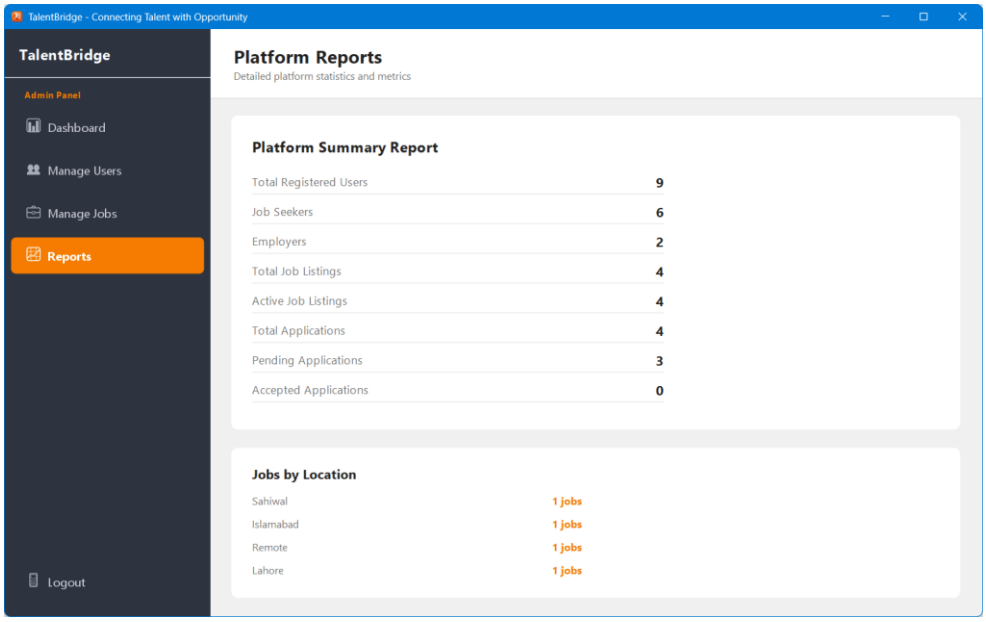
Manage Users: A centralized registry where admins can delete users, triggering cascading deletions in related tables for data consistency.



Manage Jobs: Allows moderation of job listings, ensuring only appropriate content remains on the platform.



Reports Tab: It allows to view a brief report of system users, jobs, location, applications etc.



5. Testing

The application was manually tested by exercising each feature end-to-end against the live MySQL database:

- **Login validation:** Correct credentials for Admin, Employer, and Job Seeker roles successfully grant access, while invalid or incorrect credentials are appropriately rejected.
- **Job search & filtering:** Searching for jobs by keywords and filtering by job type (e.g., Full-Time, Remote) and location returns the expected subset of active job listings.
- **Application flow:** Applying for a job and submitting a cover letter correctly creates a new application record in the database, linking the Job Seeker to the specific Job and Employer.
- **Application management & withdrawal:** Job Seekers successfully withdrawing an application updates its status to "Withdrawn," while Employers updating an application's status (e.g., Shortlisted, Accepted, Rejected) reflects immediately across both users' dashboards.
- **Role-specific CRUD operations:** Employers successfully adding, editing, and toggling the active status of their job listings, as well as Admins deleting users or jobs, are reflected immediately in the respective database tables.
- **Role-based navigation:** Confirmed that Admin, Employer, and Job Seeker sessions securely route to their specific UI dashboards and only expose their own respective menus, tabs, and capabilities.

6. Conclusion

The **Talent Bridge** project successfully demonstrates the practical application of Object-Oriented Programming principles in designing a functional, multi-role software solution. By creating distinct modules for Job Seekers, Employers, and Administrators, the application effectively addresses the core requirements of a modern employment portal. The strict adherence to OOP concepts such as utilizing Inheritance for role management, Polymorphism for dynamic dashboard rendering, and the Singleton pattern for database connectivity resulted in a modular, scalable, and highly maintainable codebase.

Furthermore, the project proved that a rich, engaging desktop user experience could be achieved natively without relying on external design libraries, thanks to the custom 2D rendering capabilities of Java Swing and AWT. The seamless integration with a normalized MySQL database ensures that all user profiles, job listings, and application pipelines are managed with high reliability and data integrity. Ultimately, Talent Bridge not only fulfills its primary objective of connecting talent with opportunity but also stands as a comprehensive showcase of strong software engineering practices, robust database management, and creative UI design using Core Java.

7. Future Enhancements

Web & Mobile Migration:

Transition the application from a monolithic desktop client to a RESTful API (e.g., using Spring Boot) paired with a modern web or mobile frontend (like React or Angular) to allow access from any device without local installation.

In-App Communication:

Develop a real-time messaging system within the portal to allow Employers and Job Seekers to communicate directly, schedule interviews, and clarify application details seamlessly.

Enhanced Security Measures:

Upgrade the platform's security by implementing industry-standard password hashing algorithms (such as BCrypt) instead of storing passwords in plain text, and introduce email-based two-factor authentication (2FA).

Resume Parsing & Generation:

Allow candidates to upload a PDF resume that automatically populates their digital profile, or conversely, allow them to export their digital profile as a neatly formatted PDF resume.